

Playbook de simulation et démonstration — Validation Phase 5 Pi-CloudDOOM

Objectif du document

Ce document sert de guide pratique pour simuler et présenter le déploiement du projet **Pi-CloudDOOM** devant le jury. Il décrit exactement :

- quoi montrer ;
- dans quel ordre ;
- quelles commandes exécuter ;
- quels résultats attendre ;
- quoi dire au jury ;
- quels cas d'usage tester ;
- comment réagir si une partie ne fonctionne pas ;
- comment prouver que l'équipe comprend réellement le déploiement.

Ce document est différent du rapport technique. Le rapport technique explique ce qui a été fait. Ce playbook explique **comment le démontrer en direct**.

1. Objectif global de la démonstration

L'objectif de la démonstration n'est pas uniquement de montrer l'interface web. Il faut montrer au jury que l'application est réellement déployée dans un environnement cloud-native et que l'équipe maîtrise :

- l'infrastructure OpenStack ;
- le cluster Kubernetes ;
- les pods et services ;
- les images Docker ;
- l'exposition publique ;
- l'authentification Keycloak/OAuth ;
- les fonctionnalités métier ;
- l'upload et l'analyse de CV ;
- les logs ;
- le monitoring de base ;
- le diagnostic des incidents.

Le fil conducteur doit être :

Nous allons partir de l'infrastructure, montrer que le cluster est actif, montrer que nos services tournent, ouvrir l'application publique, tester des fonctionnalités réelles, puis montrer comment nous surveillons et diagnostiquons le système.

2. Préparation avant d'entrer devant le jury

2.1 Matériel à préparer

Avant la validation, préparer :

- un ordinateur avec terminal prêt ;
- accès SSH au control plane Kubernetes ;
- navigateur ouvert ;
- domaine public de l'application prêt ;
- compte utilisateur de test ;
- compte OAuth si nécessaire ;
- CV PDF de test ;
- screenshots de secours ;
- dépôt GitHub ouvert ;
- Docker Hub ouvert ;
- Keycloak admin console si possible ;
- Cloudflare dashboard si nécessaire.

2.2 Fenêtres à ouvrir avant la présentation

Préparer plusieurs onglets ou fenêtres :

1. Terminal connecté à `k8s-cp1`.
2. Navigateur sur l'application frontend.
3. Navigateur sur Keycloak admin console.
4. Navigateur sur GitHub repository.
5. Navigateur sur Docker Hub repository.
6. Dossier local contenant le CV PDF de test.
7. Screenshots de secours.

2.3 Compte de test

Préparer un compte simple :

```
Email : demo@picclouddoom.com
Password : Demo@12345
Role : ROLE_USER
```

Si ce compte n'existe pas, créer un compte équivalent avant la soutenance.

2.4 CV de test

Préparer un CV PDF simple contenant clairement :

```
Name: Demo User
Bio: Cloud and DevOps student
Skills: Java, Spring Boot, Angular, Docker, Kubernetes, OpenStack, Redis, Kafka
Education: Engineering student in Cloud Computing
Experience: Backend Developer Intern, DevOps Project Member
```

Le CV doit être léger, lisible, et ne doit pas contenir d'informations personnelles sensibles.

2.5 Commandes à garder dans un fichier texte

Créer un fichier local `demo-commands.txt` avec toutes les commandes suivantes, pour éviter les erreurs de frappe :

```
kubectl get nodes -o wide
kubectl get ns
kubectl get all -n piclouddoom
kubectl get pods -n piclouddoom -o wide
kubectl get deploy -n piclouddoom
kubectl get svc -n piclouddoom
kubectl get pvc -n piclouddoom
kubectl get storageclass
kubectl get events -n piclouddoom --sort-by=.lastTimestamp
kubectl get deploy user-service -n piclouddoom -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
kubectl logs deployment/user-service -n piclouddoom --tail=80
kubectl logs deployment/keycloak -n piclouddoom --tail=80
kubectl rollout status deployment/user-service -n piclouddoom --timeout=180s
```

3. Répartition des rôles pendant la démo

L'équipe contient 7 membres. Pendant la démonstration, chaque membre doit avoir une tâche précise.

Membre	Rôle pendant la démo	Responsabilité
Membre 1	Animateur principal	Introduit la démonstration et fait les transitions
Membre 2	Architecture	Explique les composants et le schéma global

Membre	Rôle pendant la démo	Responsabilité
Membre 3	Docker	Montre GitHub/Docker Hub/images
Membre 4	Kubernetes/OpenStack	Exécute les commandes cluster
Membre 5	Sécurité/OAuth	Montre Keycloak, login, OAuth
Membre 6	IA/Monitoring	Montre upload CV, logs, diagnostic
Membre 7	Backup/Conclusion	Gère les screenshots et conclut

Important : un seul membre doit manipuler le terminal principal pour éviter la confusion. Les autres expliquent.

4. Timing recommandé

Durée idéale : **15 à 20 minutes**.

Étape	Contenu	Durée
1	Introduction	1 min
2	Architecture globale	2 min
3	GitHub + Docker Hub	2 min
4	OpenStack/Kubernetes	4 min
5	Application frontend	2 min
6	Login/Keycloak/OAuth	3 min
7	Fonctionnalités métier + CV/IA	3 min
8	Logs/monitoring/diagnostic	2 min
9	Conclusion	1 min

Si le jury donne moins de temps, utiliser la version courte à la fin de ce document.

5. Scénario complet de démonstration

Étape 1 — Introduction de la démo

Qui parle ?

Membre 1.

Ce qu'il faut dire

Nous allons maintenant faire une démonstration de la phase de déploiement. L'objectif est de montrer que notre application Pi-CloudDOOM n'est pas seulement lancée localement, mais qu'elle est conteneurisée, déployée sur Kubernetes, hébergée sur OpenStack, exposée publiquement, sécurisée avec Keycloak, et surveillable avec des commandes de diagnostic.

Nous allons commencer par l'infrastructure et finir par les fonctionnalités utilisateur.

Action

Aucune commande. Juste poser le contexte.

Étape 2 — Montrer le dépôt GitHub

Qui parle ?

Membre 3.

Action

Ouvrir le dépôt :

```
https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM
```

Montrer :

- branches ;
- services ;
- Dockerfiles ;
- fichiers de configuration ;
- éventuellement dossier `k8s` si présent ;
- README ou structure du projet.

Ce qu'il faut dire

Ici, nous montrons le dépôt principal du projet. Le code est organisé autour d'une architecture microservices. Chaque service possède sa logique métier et peut être conteneurisé indépendamment.

La branche d'intégration finale regroupe les parties principales du projet. Le déploiement repose sur cette version intégrée.

Points à montrer rapidement

```
frontend Angular
user-service
community-service
interview-service
training-service
Dockerfiles
docker-compose.yml
configuration Kubernetes si présente
```

Résultat attendu

Le jury comprend que le projet est structuré et versionné.

Étape 3 — Montrer Docker Hub et les images

Qui parle ?

Membre 3.

Action

Ouvrir Docker Hub ou montrer les images locales.

Commande possible :

```
docker images | grep pi-clouddoom
```

Ou montrer l'image utilisée par Kubernetes plus tard avec :

```
kubectl get deploy user-service -n piclouddoom -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

Ce qu'il faut dire

Après le développement, chaque service a été transformé en image Docker. Kubernetes ne déploie pas directement le code source, il déploie des images Docker versionnées.

Ces images sont publiées sur Docker Hub pour que les nœuds Kubernetes puissent les récupérer.

Exemple à citer

```
docker.io/azizbna/pi-clouddoom-user-service:cv-ollama-fast-v2-v2
```

Résultat attendu

Le jury voit le lien entre code source, image Docker et déploiement Kubernetes.

Étape 4 — Montrer le cluster Kubernetes

Qui parle ?

Membre 4.

Action

Dans le terminal connecté au control plane :

```
kubectl get nodes -o wide
```

Résultat attendu

Voir les nœuds du cluster, par exemple :

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
k8s-cp1	Ready	control-plane	...	...	...
k8s-w1	Ready	<none>	...	...	...
k8s-w2	Ready	<none>	...	...	...
k8s-w3	Ready	<none>	...	...	...
k8s-w4	Ready	<none>	...	...	...

Ce qu'il faut dire

Cette commande montre que notre cluster Kubernetes est actif. Le nœud control plane pilote le cluster, et les workers exécutent les pods applicatifs.

Ce cluster tourne sur des machines virtuelles OpenStack. OpenStack fournit donc l'infrastructure, et Kubernetes orchestre les conteneurs au-dessus de cette infrastructure.

Si un nœud n'est pas Ready

Dire :

Ici, nous voyons qu'un nœud n'est pas Ready. C'est justement l'intérêt de Kubernetes : nous pouvons diagnostiquer l'état du cluster, identifier quel nœud pose problème, et vérifier si les pods sont encore disponibles sur les autres nœuds.

Puis lancer :

```
kubectl describe node <node-name>
```

Étape 5 — Montrer le namespace du projet

Qui parle ?

Membre 4.

Action

```
kubectl get ns
```

Puis :

```
kubectl get all -n picloudboom
```

Ce qu'il faut dire

Nous avons isolé les ressources du projet dans le namespace `picloudboom`. Cela permet d'organiser les ressources Kubernetes et d'éviter les conflits avec d'autres composants du cluster.

La commande `kubectl get all` affiche les pods, services, deployments et replicasets du projet.

Résultat attendu

Le jury voit les ressources Kubernetes du projet.

Étape 6 — Montrer les pods

Qui parle ?

Membre 4.

Action

```
kubectl get pods -n piclouddoom -o wide
```

Résultat attendu

Voir des pods comme :

```
user-service-xxxxx      1/1 Running
community-service-xxxxx 1/1 Running
interview-service-xxxxx 1/1 Running
training-service-xxxxx  1/1 Running
keycloak-xxxxx          1/1 Running
postgres-xxxxx          1/1 Running
redis-xxxxx             1/1 Running
kafka-xxxxx             1/1 Running
```

Ce qu'il faut dire

Cette commande montre les pods qui exécutent les composants de notre application. La colonne READY indique si le conteneur est prêt. La colonne STATUS indique s'il est Running, Pending, CrashLoopBackOff ou en erreur.

Pendant le projet, nous avons utilisé cette commande très souvent pour diagnostiquer l'état réel du déploiement.

Si un pod est en erreur

Ne pas paniquer. Dire :

Ce cas est intéressant parce qu'il permet de montrer notre démarche de diagnostic. Nous allons vérifier les événements et les logs du pod.

Puis :

```
kubectl describe pod <pod-name> -n piclouddoom
kubectl logs <pod-name> -n piclouddoom --tail=80
```

Étape 7 — Montrer les services Kubernetes

Qui parle ?

Membre 4.

Action

```
kubectl get svc -n piclouddoom
```

Ce qu'il faut dire

Les Services Kubernetes permettent d'exposer les pods avec un nom stable. Même si un pod est recréé avec une nouvelle IP, le Service garde le même nom DNS interne.

Par exemple, les microservices ne doivent pas appeler PostgreSQL avec `localhost`. Ils doivent utiliser le nom du service Kubernetes, comme `postgres`, `redis`, `kafka` ou `keycloak` selon la configuration.

Point important

Insister sur :

Dans Kubernetes, `localhost` signifie le conteneur lui-même, pas les autres services.

Étape 8 — Montrer les Deployments

Qui parle ?

Membre 4.

Action

```
kubectl get deploy -n piclouddoom
```

Ce qu'il faut dire

Les Deployments définissent l'état souhaité de nos applications. Par exemple, si nous voulons un replica du user-service, Kubernetes vérifie que ce replica existe et le recrée s'il tombe.

Les Deployments sont aussi utilisés pour les rollouts et rollbacks.

Action complémentaire

```
kubectl rollout status deployment/user-service -n picloudoom --timeout=180s
```

Ce qu'il faut dire

Cette commande vérifie si le dernier déploiement du user-service est terminé correctement.

Étape 9 — Montrer l'image Docker réellement utilisée

Qui parle ?

Membre 3 ou 4.

Action

```
kubectl get deploy user-service -n picloudoom -o  
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

Résultat attendu

Exemple :

```
docker.io/azizbna/pi-clouddoom-user-service:cv-ollama-fast-v2-v2
```

Ce qu'il faut dire

Cette commande prouve quelle image Docker est réellement utilisée par Kubernetes. C'est important car parfois l'erreur vient d'un tag incorrect ou d'une image qui n'existe pas dans Docker Hub.

Nous avons rencontré un cas `ErrImagePull`, et ce type de commande nous a permis de vérifier exactement l'image demandée par Kubernetes.

Étape 10 — Montrer la persistance

Qui parle ?

Membre 4.

Action

```
kubectl get storageclass  
kubectl get pvc -n piclouddoom
```

Ce qu'il faut dire

Certains composants comme PostgreSQL ont besoin de persistance. Sans volume persistant, les données peuvent être perdues si le pod redémarre.

Au départ, nous n'avions pas de StorageClass disponible. Nous avons installé `local-path-provisioner` pour permettre la création de volumes persistants locaux.

Résultat attendu

Voir une StorageClass comme :

```
local-path
```

Et des PVC en état :

```
Bound
```

Si aucun PVC n'est affiché

Dire :

Selon la configuration finale, certains services peuvent être lancés sans PVC pour la démonstration. Mais dans une configuration complète, les bases et les composants persistants doivent utiliser des PVC.

Étape 11 — Ouvrir l'application publique

Qui parle ?

Membre 1 ou 5.

Action

Ouvrir le domaine public de l'application dans le navigateur.

Ce qu'il faut dire

Maintenant que nous avons montré le déploiement côté infrastructure, nous allons montrer l'application côté utilisateur.

L'accès public passe par notre configuration d'exposition, notamment Cloudflare pour le DNS et l'accès externe.

Résultat attendu

Le frontend Angular s'affiche.

Si le frontend ne s'ouvre pas

Utiliser le plan de secours :

```
kubectl get pods -n piclouddoom  
kubectl get svc -n piclouddoom
```

Puis dire :

Si l'interface publique ne répond pas, le problème peut venir de l'exposition externe, du DNS, du tunnel ou de Cloudflare. Nous pouvons prouver que les services sont déployés côté cluster et montrer les screenshots de fonctionnement préparés.

Étape 12 — Tester le login classique

Qui parle ?

Membre 5.

Action

Dans l'application :

1. cliquer sur Login ;
2. redirection vers Keycloak ;
3. entrer le compte de test ;
4. valider ;
5. revenir vers l'application.

Ce qu'il faut dire

L'authentification est centralisée avec Keycloak. Le frontend ne gère pas directement les mots de passe. Il redirige vers Keycloak, qui authentifie l'utilisateur et renvoie un token.

Les microservices Spring Boot valident ensuite ce token JWT pour sécuriser leurs endpoints.

Résultat attendu

L'utilisateur est connecté et arrive dans le dashboard/profil.

Si login échoue

Dire :

Les erreurs de login viennent souvent d'un problème de redirect URI ou de configuration Keycloak. Nous allons montrer les logs Keycloak.

Commande :

```
kubectl logs deployment/keycloak -n piclouddoom --tail=100
```

Étape 13 — Montrer Keycloak

Qui parle ?

Membre 5.

Action

Ouvrir Keycloak admin console.

Montrer :

- realm `myapp-realm` ;
- client frontend, par exemple `angular-client` ;
- rôles ;
- providers Google/LinkedIn/GitHub ;
- redirect URIs.

Ce qu'il faut dire

Keycloak joue le rôle de serveur d'identité. Il gère les utilisateurs, les rôles, les tokens et les providers externes.

Les rôles de sécurité comme `ROLE_USER`, `ROLE_ADMIN` et `ROLE_MANAGER` sont utilisés par Spring Security pour autoriser ou refuser l'accès aux endpoints.

Point important

Dire :

Les rôles Keycloak ne doivent pas être confondus avec les statuts métier comme `FREE` ou `PREMIUM`. Les rôles Keycloak servent à la sécurité, les statuts métier servent à la logique fonctionnelle de l'application.

Étape 14 — Tester OAuth avec un provider

Qui parle ?

Membre 5.

Action

Tester un provider qui fonctionne le mieux pour la demo, par exemple LinkedIn si c'est le plus stable.

1. logout ;
2. cliquer sur Continue with LinkedIn / Google / GitHub ;
3. redirection provider ;
4. retour application.

Ce qu'il faut dire

Nous avons configuré plusieurs providers OAuth : Google, LinkedIn et GitHub. Cette partie est sensible car chaque provider exige une redirect URI exacte.

Pendant le déploiement, nous avons corrigé des problèmes liés à ces redirect URIs, car les URLs locales ne sont pas les mêmes que les URLs publiques.

Si OAuth ne fonctionne pas en direct

Ne pas perdre trop de temps. Dire :

Si l'OAuth échoue pendant la démonstration, cela prouve justement la sensibilité de cette partie. Les providers OAuth vérifient strictement les domaines, les protocoles HTTP/HTTPS et les callbacks. Nous avons préparé la configuration et nous pouvons montrer les redirect URIs dans Keycloak et dans le provider.

Puis montrer Keycloak au lieu d'insister sur le navigateur.

Étape 15 — Tester le profil utilisateur

Qui parle ?

Membre 6.

Action

Après login, ouvrir la page profil.

Ce qu'il faut dire

Cette page récupère les données utilisateur depuis le backend, notamment depuis le user-service. Cela montre que le frontend communique bien avec les microservices déployés.

Résultat attendu

Informations utilisateur affichées.

Commande logs en parallèle

Dans le terminal :

```
kubectll logs deployment/user-service -n picloudoom -f
```

Puis rafraîchir le profil.

Ce qu'il faut dire

Ici, nous observons les logs du user-service pendant que nous utilisons l'application. Cela montre le lien entre l'action frontend et le traitement backend.

Pour arrêter : `Ctrl + C`.

Étape 16 — Tester l'upload CV

Qui parle ?

Membre 6.

Action

1. Aller dans le profil ou la page CV.
2. Cliquer sur upload CV.

3. Sélectionner le PDF de test.
4. Valider.
5. Observer la réponse.

Ce qu'il faut dire

Cette fonctionnalité est importante parce qu'elle combine plusieurs parties du projet : frontend Angular, user-service Spring Boot, stockage du fichier, extraction de texte et enrichissement IA.

Le CV est stocké côté backend, puis une URL relative est conservée pour permettre l'action View CV.

Résultat attendu

- upload réussi ;
- message de succès ;
- CV visible ;
- données extraites si IA active.

Logs à montrer

```
kubectl logs deployment/user-service -n picloudboom --tail=100
```

Étape 17 — Tester View CV

Qui parle ?

Membre 6.

Action

Cliquer sur [View CV](#).

Ce qu'il faut dire

Le bouton View CV montre que le fichier uploadé est bien accessible après stockage. Pendant le projet, nous avons corrigé cette partie pour que l'URL du CV soit correctement construite et accessible côté frontend.

Résultat attendu

Le PDF s'ouvre ou se télécharge.

Si cela échoue

Dire :

Si le lien échoue, il faut vérifier le chemin statique exposé par le backend, l'URL sauvegardée dans le profil et la configuration frontend. C'est un problème de mapping fichier/URL, pas seulement un problème d'upload.

Étape 18 — Tester le parsing IA du CV

Qui parle ?

Membre 6.

Action

Après upload, montrer les champs enrichis :

- bio ;
- skills ;
- education ;
- experiences.

Ce qu'il faut dire

Après l'upload, le texte du CV est extrait et envoyé à un modèle IA pour retourner une structure JSON. Ensuite, le user-service sauvegarde ces informations dans le profil utilisateur.

Nous avons étudié Gemini au départ, puis nous avons migré vers Ollama pour mieux contrôler le traitement et éviter certaines contraintes d'API ou de quota.

Exemple de résultat attendu

```
Skills: Java, Spring Boot, Angular, Docker, Kubernetes, OpenStack
Bio: Cloud and DevOps student
Education: Engineering student in Cloud Computing
```

Si l'IA est lente ou indisponible

Dire :

L'IA est le composant le plus lourd. C'est pour cette raison que nous avons proposé une stratégie hybride : garder l'application principale sur OpenStack/Kubernetes et déployer le service IA séparément, par exemple sur Azure Container Apps.

La fonctionnalité est conçue pour être appelée à la demande, au moment de l'upload CV, pas en permanence.

Étape 19 — Montrer les logs applicatifs

Qui parle ?

Membre 6 ou 7.

Action

```
kubectl logs deployment/user-service -n picloudoom --tail=80
```

Puis :

```
kubectl logs deployment/keycloak -n picloudoom --tail=80
```

Ce qu'il faut dire

Les logs sont essentiels pour comprendre ce qui se passe dans une architecture microservices. En cas de problème, nous ne devinons pas : nous lisons les logs, les events et la description des pods.

Résultat attendu

Afficher des logs récents sans erreur critique.

Étape 20 — Montrer les événements Kubernetes

Qui parle ?

Membre 7.

Action

```
kubectl get events -n picloudoom --sort-by=.lastTimestamp
```

Ce qu'il faut dire

Les events Kubernetes permettent d'identifier les erreurs d'image, les problèmes de scheduling, les redémarrages et les erreurs de pull. Par exemple, quand nous avons

rencontré `ErrImagePull`, les events nous ont aidés à comprendre que Kubernetes ne pouvait pas récupérer l'image.

Résultat attendu

Afficher les événements récents.

Étape 21 — Simuler un diagnostic sans casser le système

Objectif

Montrer au jury que l'équipe sait diagnostiquer sans provoquer une vraie panne.

Option 1 — Diagnostic d'un ancien problème `ErrImagePull`

Ne pas casser le Deployment en direct. Expliquer avec la commande :

```
kubectl get deploy user-service -n piclouddoom -o  
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

Puis dire :

Si cette image avait un mauvais tag, Kubernetes afficherait `ErrImagePull`. La première chose à faire est de vérifier le tag exact utilisé par le Deployment, puis de consulter les events avec `kubectl describe pod`.

Option 2 — Diagnostic avec un pod test

Créer un pod temporaire pour tester DNS :

```
kubectl run test-dns -n piclouddoom --image=busybox:1.36 --rm -it -- nslookup  
user-service
```

Ce qu'il faut dire

Ce test permet de vérifier que le service Kubernetes est résolu par DNS à l'intérieur du cluster.

Option 3 — Tester Redis sans casser le système

```
kubectl exec -it deployment/redis -n piclouddoom -- redis-cli ping
```

Résultat attendu :

PONG

Ce qu'il faut dire

Ici, nous vérifions que Redis répond correctement depuis son conteneur.

6. Cas d'usage fonctionnels à montrer

Use case 1 — Utilisateur se connecte

Objectif

Montrer l'authentification avec Keycloak.

Étapes

1. Ouvrir l'application.
2. Cliquer Login.
3. Redirection Keycloak.
4. Entrer email/password.
5. Retour dashboard.

Résultat attendu

Utilisateur connecté.

Ce que cela prouve

- frontend accessible ;
 - Keycloak fonctionne ;
 - redirect URI correcte ;
 - token reçu ;
 - session utilisateur active.
-

Use case 2 — Utilisateur consulte son profil

Objectif

Montrer la communication frontend/backend.

Étapes

1. Aller sur Profil.
2. Rafraîchir la page.
3. Montrer logs user-service.

Résultat attendu

Données utilisateur chargées.

Ce que cela prouve

- frontend Angular appelle user-service ;
 - user-service répond ;
 - base de données disponible ;
 - token accepté.
-

Use case 3 — Utilisateur upload un CV

Objectif

Montrer une fonctionnalité métier complète.

Étapes

1. Sélectionner CV PDF.
2. Upload.
3. Attendre réponse.
4. Vérifier profil.

Résultat attendu

CV sauvegardé.

Ce que cela prouve

- upload multipart fonctionne ;
 - backend reçoit le fichier ;
 - stockage fonctionne ;
 - profil mis à jour.
-

Use case 4 — Utilisateur visualise son CV

Objectif

Montrer l'accès au fichier stocké.

Étapes

1. Cliquer View CV.
2. Ouvrir PDF.

Résultat attendu

PDF accessible.

Ce que cela prouve

- URL CV correcte ;
 - mapping statique backend correct ;
 - frontend utilise le bon chemin.
-

Use case 5 — Enrichissement IA du profil

Objectif

Montrer la valeur ajoutée IA.

Étapes

1. Upload CV contenant des compétences claires.
2. Attendre parsing.
3. Afficher skills/bio/education.

Résultat attendu

Profil enrichi automatiquement.

Ce que cela prouve

- extraction texte ;
 - appel IA ;
 - parsing JSON ;
 - sauvegarde dans user-service ;
 - affichage frontend.
-

Use case 6 — Diagnostic d'un service

Objectif

Montrer la compétence DevOps.

Étapes

1. Lancer `kubectl get pods`.
2. Lancer `kubectl logs`.
3. Lancer `kubectl describe` si besoin.
4. Expliquer comment trouver la cause.

Résultat attendu

Le jury voit une démarche claire.

Ce que cela prouve

- maîtrise Kubernetes ;
- capacité de troubleshooting ;
- compréhension de l'état réel du déploiement.

7. Tests techniques prêts à exécuter

Test 1 — Cluster actif

```
kubectl get nodes -o wide
```

Attendu : nodes `Ready`.

Test 2 — Ressources projet

```
kubectl get all -n picloudboom
```

Attendu : pods, services, deployments visibles.

Test 3 — Pods Running

```
kubectl get pods -n picloudboom -o wide
```


Attendu : pods en `Running` ou explication claire pour ceux qui ne le sont pas.

Test 4 — Services exposés en interne

```
kubectl get svc -n piclouddoom
```

Attendu : services pour frontend/backend/db/keycloak/etc.

Test 5 — Image Docker exacte

```
kubectl get deploy user-service -n piclouddoom -o  
jsonpath='{.spec.template.spec.containers[0].image}'
```

Attendu : image Docker Hub correcte.

Test 6 — Logs user-service

```
kubectl logs deployment/user-service -n piclouddoom --tail=80
```

Attendu : logs backend.

Test 7 — Logs Keycloak

```
kubectl logs deployment/keycloak -n piclouddoom --tail=80
```

Attendu : logs Keycloak.

Test 8 — Events Kubernetes

```
kubectl get events -n piclouddoom --sort-by=.lastTimestamp
```

Attendu : événements récents.

Test 9 — Storage

```
kubectl get storageclass  
kubectl get pvc -n piclouddoom
```

Attendu : StorageClass disponible, PVC Bound si utilisés.

Test 10 — Rollout status

```
kubectl rollout status deployment/user-service -n piclouddoom --timeout=180s
```

Attendu : rollout terminé ou état clair.

8. Questions du jury pendant la démo — réponses rapides

Question : Pourquoi vous montrez le terminal au lieu de seulement l'application ?

Réponse :

Parce que la phase évaluée est le déploiement. L'interface montre que l'application fonctionne, mais le terminal prouve qu'elle tourne réellement dans Kubernetes avec des pods, services, images Docker, logs et ressources cloud.

Question : Comment savez-vous que ce n'est pas lancé localement ?

Réponse :

Nous montrons les nœuds Kubernetes, le namespace, les pods, les services et l'image Docker utilisée par le Deployment. Cela prouve que l'application est orchestrée dans le cluster.

Question : Pourquoi ne pas utiliser seulement Docker Compose ?

Réponse :

Docker Compose est utile pour le développement local. Kubernetes est plus adapté à une architecture microservices déployée dans un environnement cloud, avec rollouts, services internes, self-healing, volumes et observabilité.

Question : Qu'est-ce qui prouve que l'image Docker est bien celle déployée ?

Réponse :

La commande `kubectl get deploy ... jsonpath` affiche directement l'image configurée dans le Deployment Kubernetes. C'est la source de vérité côté cluster.

Question : Que faites-vous si un pod tombe ?

Réponse :

Nous vérifions d'abord `kubectl get pods`, puis `kubectl describe pod`, ensuite `kubectl logs`. Selon l'erreur, nous corrigeons l'image, la configuration, la connexion à la base ou les variables d'environnement, puis nous relançons le rollout.

Question : Pourquoi Keycloak ?

Réponse :

Keycloak centralise l'authentification, les rôles, les tokens JWT et les providers OAuth. Cela évite de coder manuellement toute la sécurité et permet une intégration propre avec Angular et Spring Boot.

Question : Pourquoi Cloudflare ?

Réponse :

Cloudflare permet de gérer le domaine public, le DNS, HTTPS et l'accès externe. Il est aussi important pour OAuth, car les providers exigent des URLs publiques exactes.

Question : Pourquoi l'IA n'est pas forcément dans le même cluster ?

Réponse :

Les modèles IA comme Ollama consomment beaucoup de ressources. Comme notre cluster OpenStack est limité, nous avons proposé une stratégie hybride : garder l'application principale dans Kubernetes/OpenStack et exécuter l'IA séparément sur Azure si nécessaire.

9. Version courte de la démonstration en 7 minutes

Si le jury donne très peu de temps, suivre cette version.

9.1 Montrer cluster

```
kubectl get nodes -o wide
```

Phrase :

Notre cluster Kubernetes est actif sur OpenStack.

9.2 Montrer application déployée

```
kubectl get pods -n piclouddoom -o wide
```

Phrase :

Les composants de l'application tournent sous forme de pods.

9.3 Montrer services

```
kubectl get svc -n piclouddoom
```

Phrase :

Les services Kubernetes permettent la communication entre les composants.

9.4 Montrer image Docker

```
kubectl get deploy user-service -n piclouddoom -o  
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

Phrase :

Cette image Docker est celle réellement déployée par Kubernetes.

9.5 Ouvrir frontend

Ouvrir application.

Phrase :

L'application est accessible publiquement.

9.6 Login

Faire login rapide.

Phrase :

L'authentification est gérée par Keycloak.

9.7 Logs

```
kubectl logs deployment/user-service -n piclouddoom --tail=50
```

Phrase :

Nous pouvons diagnostiquer l'application avec les logs Kubernetes.

9.8 Conclusion

Phrase :

Cette démonstration prouve la containerisation, l'orchestration, l'exposition, la sécurité et la capacité de diagnostic.

10. Version longue de la démonstration en 20 minutes

Ordre :

1. Introduction.
2. GitHub repo.
3. Docker Hub/images.
4. OpenStack infrastructure explanation.
5. `kubectl get nodes`.
6. `kubectl get all -n piclouddoom`.
7. `kubectl get pods`.
8. `kubectl get svc`.
9. `kubectl get deploy`.
10. Image Docker exacte.
11. StorageClass/PVC.
12. Application frontend.
13. Login Keycloak.

14. Keycloak admin console.
 15. OAuth provider.
 16. Profil utilisateur.
 17. Upload CV.
 18. View CV.
 19. Parsing IA.
 20. Logs user-service.
 21. Events Kubernetes.
 22. Conclusion.
-

11. Plan de secours complet

Situation 1 — Internet ne fonctionne pas

Montrer screenshots préparés :

- frontend ;
- pods ;
- services ;
- Docker Hub ;
- Keycloak ;
- upload CV.

Dire :

Nous avons préparé des captures de secours, mais nous pouvons toujours expliquer la démarche technique et les commandes utilisées.

Situation 2 — Cloudflare ne répond pas

Montrer :

```
kubectl get pods -n picloudboom  
kubectl get svc -n picloudboom
```

Dire :

L'application peut être saine dans Kubernetes même si l'exposition externe a un problème.
Dans ce cas, nous diagnostiquons la couche Cloudflare/DNS/tunnel.

Situation 3 — Un pod est en CrashLoopBackOff

Montrer :

```
kubectl logs <pod-name> -n piclouddoom --previous
kubectl describe pod <pod-name> -n piclouddoom
```

Dire :

CrashLoopBackOff signifie que le conteneur démarre puis s'arrête. La cause est souvent dans les logs : variable manquante, erreur DB, erreur Keycloak ou mémoire insuffisante.

Situation 4 — ErrImagePull

Montrer :

```
kubectl describe pod <pod-name> -n piclouddoom
kubectl get deploy <deployment-name> -n piclouddoom -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

Dire :

ErrImagePull signifie que Kubernetes n'arrive pas à récupérer l'image. Nous vérifions le tag, Docker Hub, la visibilité de l'image et les credentials si l'image est privée.

Situation 5 — Login OAuth échoue

Dire :

OAuth dépend strictement des redirect URIs. Il faut vérifier le provider, Keycloak, Cloudflare et le domaine public.

Montrer :

- Keycloak client ;
- Identity Provider ;
- redirect URI ;
- logs Keycloak.

Situation 6 — IA lente

Dire :

Le modèle IA consomme beaucoup de ressources. C'est pour cela que nous avons proposé une stratégie hybride avec Azure Container Apps, afin de ne pas surcharger le cluster OpenStack.

12. Captures de secours à préparer

Préparer un dossier nommé :

```
validation-phase5-screenshots
```

Contenu recommandé :

```
01-kubectl-get-nodes.png
02-kubectl-get-pods.png
03-kubectl-get-svc.png
04-kubectl-get-deploy.png
05-user-service-image.png
06-dockerhub-user-service.png
07-github-repo.png
08-keycloak-realm.png
09-keycloak-client.png
10-oauth-providers.png
11-cloudflare-dns.png
12-frontend-home.png
13-login-page.png
14-profile-page.png
15-upload-cv.png
16-view-cv.png
17-ai-skills-result.png
18-user-service-logs.png
19-storageclass-pvc.png
20-events-kubernetes.png
```

Ces screenshots peuvent sauver la validation si la connexion ou l'environnement tombe.

13. Script final à lire si le jury demande une synthèse

Pour résumer, notre démonstration suit le chemin complet d'un déploiement cloud-native. Nous avons d'abord montré le code et les images Docker, ensuite le cluster Kubernetes hébergé sur OpenStack, puis les pods, services, deployments, volumes et logs. Ensuite, nous avons montré l'application accessible publiquement, l'authentification avec Keycloak, les providers OAuth, puis une fonctionnalité métier comme l'upload et l'analyse de CV.

Cette démonstration prouve que nous comprenons non seulement comment lancer l'application, mais aussi comment la diagnostiquer, la surveiller et justifier les choix techniques. Docker standardise les services, Kubernetes les orchestre, OpenStack fournit l'infrastructure, Cloudflare expose l'application, Keycloak sécurise l'accès, et la stratégie hybride permet de gérer les composants lourds comme l'IA.

14. Checklist minute avant passage

Avant d'entrer :

- ☐ Terminal connecté au cluster.
 - ☐ `kubectl get nodes` testé.
 - ☐ `kubectl get pods -n piclouddoom` testé.
 - ☐ Application ouverte dans le navigateur.
 - ☐ Compte de test prêt.
 - ☐ CV PDF prêt.
 - ☐ Keycloak admin prêt si possible.
 - ☐ GitHub ouvert.
 - ☐ Docker Hub ouvert.
 - ☐ Screenshots prêts.
 - ☐ Chaque membre connaît sa phrase principale.
-

15. Les phrases importantes à ne pas oublier

- Nous ne montrons pas seulement une interface, nous montrons une chaîne complète de déploiement.
 - Docker nous donne des images reproductibles.
 - Kubernetes orchestre les conteneurs et maintient l'état souhaité.
 - OpenStack fournit l'infrastructure cloud privée.
 - Cloudflare permet l'accès public et aide à aligner les URLs OAuth.
 - Keycloak centralise l'identité, les rôles et les tokens.
 - Les logs et events Kubernetes sont notre première méthode de diagnostic.
 - L'IA avec Ollama est lourde, donc la stratégie hybride avec Azure est justifiée.
 - La vraie valeur de cette phase est la capacité à déployer, observer, diagnostiquer et expliquer.
-

16. Conclusion du playbook

Ce playbook doit être utilisé comme guide direct pour la validation. Il permet à l'équipe de présenter une démonstration structurée, sans improvisation excessive.

La meilleure démonstration est celle qui suit une logique claire :

1. infrastructure ;
2. cluster ;
3. images ;
4. pods ;
5. services ;
6. exposition ;
7. sécurité ;
8. fonctionnalités ;
9. logs ;
10. diagnostic ;
11. conclusion.

En suivant ce scénario, l'équipe montre qu'elle maîtrise la phase de déploiement au niveau technique, fonctionnel et DevOps.